

|               |                        |                  |       |
|---------------|------------------------|------------------|-------|
| Группа        | <b>Z3342</b>           | К работе допущен | _____ |
| Студент       | <b>Погорелко В. А.</b> | Работа выполнена | _____ |
| Преподаватель | <b>Маслов И. Д.</b>    | Отчёт принят     | _____ |

## **Рабочий протокол и отчёт по лабораторной работе №3**

### **[C++ & UNIX]: C++ CLI / FUNCTION / LOOP / RECURSION**

## **1. Цель работы**

Познакомить студента с основными алгоритмическими конструкциями, которые будут использоваться для создания CLI программы. Далее продемонстрировать эффективность использования механизма рекурсии.

C++ алгоритмы: CLI Калькулятор вещественных чисел +, −, ∧. Реализация с использованием только функций, условий, циклов, + и −. Вид команд в консоли: calc / plus / minus / power; Ханойская башня, результат корректной последовательности.

## **2. Задачи**

1. (C++ CLI CALC) Создать программу CALC с интерфейсом CLI Создать программу под названием CALC, которая будет принимать на вход 3 аргумента (2 операнда и 1 оператор). Оператор может быть: +, −, ∧. Реализация операторов только с использованием функций, условий, циклов, +, − и \*.
2. (C++ RECURSION) Решить задачу ханойской башни с использованием рекурсии Описание: Ханойская башня является одной из популярных головоломок XIX века. Даны три стержня, на один из которых нанизаны восемь колец, причём кольца отличаются размером и лежат меньшее на большем. Задача состоит в том, чтобы перенести пирамиду из восьми колец за наименьшее число ходов на другой стержень. За один раз разрешается переносить только одно кольцо, причём нельзя класть большее кольцо на меньшее. Результат обнаруженной последовательности шагов записать в виде двусвязного списка. В конце программы сделать вывод этого списка на экран. Освободить память списка перед завершением программы.
3. (SAVE) Результат всех вышеперечисленных шагов сохранить в репозиторий (+ отчет по данной ЛР в папку doc) Фиксацию ревизий производить строго через ветку dev. С помощью скриптов накатить ревизии на stg и на prd. Скрипты разместить в корне репозитория. Также создать скрипты по возврату к виду текущей ревизии (даже если в папке имеются несохраненные изменения + новые файлы).

### 3. Решение

1. Создать программу CALC с интерфейсом CLI Создать программу под названием CALC, которая будет принимать на вход 3 аргумента (2 операнда и 1 оператор)

В папке lab\_03/src создаем файл cli\_calc.cpp:

```
cli_calc.cpp

#include <iostream>
#include <string>
#include <vector>
#include <bitset>

using namespace std;

int main(){
    double num1, num2;
    unsigned int power;
    double result = 1.0;
    char operation, status;
    do {
        ...
    } while (status == 'y' || status == 'Y');

    return 0;
}
```

Внутри цикла do { ... } делаем switch/case метод:

```

inside do { ... }
cout << "Enter first number: ";
cin >> num1;
cout << endl;
cout << "Enter operation: ";
cin >> operation;
cout << endl;
switch (operation) {
    case '+':
        cout << "Enter second number: ";
        cin >> num2;
        cout << endl;
        cout << "Result: " << num1 + num2 << endl;
        break;
    case '-':
        cout << "Enter second number: ";
        cin >> num2;
        cout << "Result: " << num1 - num2 << endl;
        break;
    case '^':
        cout << "Enter power (must be \
                \ positive integer): ";
        cin >> power;
        for (int i = 0; i < power; i++) {
            result *= num1;
        }
        cout << "Result: " << result << endl;
        break;
    default:
        cout << "No such command" << endl;
}
cout << "Print y or Y to continue:";
cin >> status;
cout << endl << endl;

```

## 2. Решить задачу ханойской башни с использованием рекурсии

В папке lab\_03/src создаем файл hanoi.cpp:

Файл hanoi.cpp

```
#include <iostream>
#include <list>
#include <string>

using namespace std;

void hanoi_towers(int quantity, int from, int to, \
\ int buf_peg, list<string>& lstOfOps)
{
    if (quantity != 0)
    {
        hanoi_towers(quantity-1, from, buf_peg, to, lstOfOps);

        string move = to_string(from) + "->" + to_string(to);
        lstOfOps.push_back(move);

        hanoi_towers(quantity-1, buf_peg, to, from, lstOfOps);
    }
}

int main() {
    ...
}
```

И соответственно внутри функции main() делаем вызовы переменных и заполнение + финальную очистку списка.

Функция main() { ... }

```
list<string> lstOfOps;  
int start_peg, destination_peg, buffer_peg, plate_quantity;  
cout << "First pillar:" << endl;  
cin >> start_peg;  
cout << "Last pillar:" << endl;  
cin >> destination_peg;  
cout << "Buffer pillar:" << endl;  
cin >> buffer_peg;  
cout << "Number of disks:" << endl;  
cin >> plate_quantity;  
hanoi_towers(plate_quantity, start_peg, \  
destination_peg, buffer_peg, lstOfOps);  
  
for (const string& op : lstOfOps) {  
    cout << op << endl;  
}  
cout << "Is list empty before clean: " << boolalpha \  
    << lstOfOps.empty() << endl;  
lstOfOps.clear();  
cout << "Is list empty after clean: " << boolalpha \  
    << lstOfOps.empty() << endl;  
return 0;
```